When Does Physics Help? Hybrid DeepONet–PINN Reconstruction of Full-Chip Thermal Maps from Sparse On-Chip Sensors

Ruchika Todi, Siya Patil, Smriti Prashanth Vasisht, Vipul Reddy Gundalu, and Rashmi R

***Abstract*—On-chip digital thermal sensors measure temperature at only a handful of points, yet thermal management needs the full map of the die. We study this virtual-sensing problem as a neural-network learning problem and ask when knowledge of the governing heat equation improves a learned reconstruction. We compare three families: a purely data-driven neural operator (DeepONet) that maps sensor readings to the full field, a physics-informed neural network (PINN) fitted per chip from its sensors and the steady heat equation alone, and two ways of combining them: physics as an extra training loss, and a validation-weighted blend of the two predictions. On 700 simulated 12×12 mm dies with 5, 8 and 10 noisy, quantised sensors placed by POD-QR, the DeepONet reaches 11.5%, 8.6% and 6.1% relative L_2 error when trained on 500 chips, and adding the heat-equation residual to its loss changes nothing measurable. The blend, by contrast, is never worse than its better component, and its benefit grows as training data shrinks: with 25 training chips it lowers the error from 12.7% to 8.8% at 8 sensors and from 9.3% to 6.4% at 10 sensors, while the PINN also recovers peak temperature 5–6 °C more accurately than the DeepONet. On real infrared maps of two commercial Intel CPUs, a hybrid that corrects the DeepONet with a heat-equation response fitted to its residual at the sensors is better than the DeepONet in 3 of 6 settings, tied in 2 and worse in 1, and a classical gappy-POD baseline remains competitive. A survey of further architectures (FNO/PINO, FBPINN, XPINN) on the same protocol confirms the pattern: physics in the loss adds little once data are plentiful. We trace this to the reference maps themselves, which leave a 33% pointwise residual in a grid discretisation of the heat equation, and conclude that physics is most valuable as a separate, data-free estimator combined with a learned operator when training data are scarce.**

***Index Terms*—Thermal map reconstruction, virtual sensing, physics-informed neural networks, DeepONet, neural operators, sparse sensors, hybrid models, data efficiency.**

I. Introduction

POWER density in modern processors keeps rising, and with it the cost of local hot spots: they shorten device lifetime, force frequency throttling, and must be found both during design validation and in the field. Processors therefore carry digital thermal sensors (DTS), but area, power and routing limit them to a few per die, so the hottest point is usually not where a sensor sits. Infrared (IR) imaging gives the full map but needs an open package and an external camera, and numerical solvers such as finite-element analysis need the power map, which is not observed at run time. Reconstructing the full thermal map from a few sensor readings, a virtual-sensing problem, is therefore a measurement problem of practical importance [1]–[3].

Two neural-network routes to this problem have emerged. *Data-driven operator learning*, for example the deep operator network (DeepONet) [4], learns the map from sensor readings to the full field from many examples and then reconstructs a new chip in one forward pass. *Physics-informed neural networks* (PINNs) [5]–[7] instead fit a network to one chip's readings while penalising the residual of the governing partial differential equation (PDE); they need no training set but must be optimised for every chip. A large literature argues that adding physics to learning improves accuracy and data efficiency [6], [8]–[10]. Whether that holds for chip thermal maps under realistic sensor noise, and *how* physics should be added, is less clear: physics can enter as an extra loss term [11], [12], or as a separate estimator whose prediction is combined with the learned one.

This paper answers these questions with controlled experiments. We use one shared protocol (data splits, sensor layout, noise model and metrics) for every model, so differences can be attributed to the model. Our research questions are:

- **RQ1:** How accurately can a neural operator reconstruct a full-chip thermal map from 5 to 10 noisy sensors?
- **RQ2:** Does physics help, and if so, in which form and in which data regime?
- **RQ3:** Do the conclusions carry over to measured IR maps of commercial processors?

Our contributions are:

1) A controlled comparison of a data-driven DeepONet, a per-chip PINN, a physics-informed DeepONet and a validation-weighted DeepONet–PINN blend for full-chip thermal reconstruction, on 700 simulated dies with a realistic sensor model (noise, quantisation, calibration offsets) and POD-QR sensor placement.
2) A data-scarcity study (25 to 500 training chips) with paired bootstrap confidence intervals, showing that the blend is never worse than its better component and that its gain over the DeepONet grows from at most 0.4 percentage points with 500 training chips to 1–4 points with 25, while physics added as a loss term gives no measurable gain.
3) A transfer study on measured IR maps of an Intel i5-3337U and i7-8650U [13], with a physics-based correction hybrid and classical baselines.

The authors are with the Department of Information Science and Engineering, B.M.S. College of Engineering, Bengaluru 560019, India. Corresponding author: [TODO: name and e-mail].

4) A survey of further physics-informed architectures (FNO/PINO, FBPINN, XPINN, and a differentiable-solver hybrid) and a diagnosis of why a PDE loss adds little here: the reference maps themselves leave a 33% pointwise residual in the discretised heat equation.

Section II reviews related work, Section III states the problem, Section IV describes the data, Section V the models, Sections VI and VII report results on simulated and real data, Section VIII surveys further architectures, and Section IX discusses limitations.

II. Related Work

A. Full-chip thermal map estimation

Early work recovered full maps from a few sensors with linear models built from simulated or measured maps: spectral or eigen-decompositions with sensor placement chosen to make the inverse well conditioned [1], [2]. Gappy proper orthogonal decomposition (POD) [14] and POD-QR sensor placement [15] are the general form of this idea. Learning-based estimators followed: a generative adversarial network trained on IR images of a commercial multi-core CPU [16], recurrent and attention models for real-time tracking from performance counters [3], and a public IR dataset of commercial CPUs, GPUs and TPUs [13], which we use here. Physics-enforced networks have also been used for fast parametric full-chip thermal analysis when the power map is known [17] and for microprocessor-heat-sink modelling [18]. Our setting differs: the power map is unknown and only sparse, noisy sensor readings are available at reconstruction time.

B. Physics-informed learning and neural operators

PINNs [5] minimise a data misfit plus the PDE residual computed by automatic differentiation, and have been applied to forward and inverse heat-transfer problems with sparse measurements [9]. They are known to be hard to optimise when loss terms compete [19], [20]. Neural operators learn maps between function spaces from data: DeepONet [4] uses a branch network for the input function and a trunk network for the query coordinate; the Fourier neural operator (FNO) [21] learns in spectral space. Both have physics-informed variants that add the PDE residual to the training loss [11], [12]. Domain-decomposition PINNs split the domain into subdomains with local networks, either with overlapping, smoothly blended windows (FBPINN) [22] or non-overlapping tiles coupled by interface losses (XPINN) [23]. Reviews of physics-informed thermal management of electronics [10] report large speed-ups over numerical solvers but rarely compare against a strong data-driven model under one protocol, which is what we do here.

III. Problem Formulation

Let $T(\mathbf{x})$ be the steady temperature on the die $\Omega = [0, L]^2$, sampled on an $H \times W$ grid. N sensors at fixed positions $\{\mathbf{x}_i\}_{i=1}^{N}$ return noisy, quantised readings y_i, and the ambient temperature T_{amb} is known. The task is to estimate $\hat{T}$ on the whole grid from $(\{\mathbf{x}_i, y_i\}, T_{\mathrm{amb}})$. The block layout and power map are *not* available.

For the simulated dies the temperature satisfies the compact two-dimensional HotSpot model [24]

$$-\nabla \cdot \big(t_{\mathrm{die}}\, k(T)\, \nabla T\big) + h_{\mathrm{eff}}\,(T - T_{\mathrm{amb}}) = q(\mathbf{x}, T), \qquad (1)$$

with adiabatic lateral edges ($\partial T/\partial n = 0$), die thickness t_{die}, an effective vertical conductance h_{eff} to the heat sink, a temperature-dependent silicon conductivity $k(T)$ [25], and a power density q made of block-wise dynamic power plus leakage that grows as $\exp(\beta(T - 25\,^{\circ}\mathrm{C}))$. With $\theta = (T - T_{\mathrm{amb}})/S$ for a temperature scale S and coordinates scaled to the unit square, (1) becomes

$$-\varepsilon\, \nabla \cdot \big(k(T) \nabla \theta\big) + \theta = \tilde{q}, \qquad \varepsilon = \frac{t_{\mathrm{die}}}{h_{\mathrm{eff}} L^2} \approx 5.6 \times 10^{-4}, \quad (2)$$

a screened Poisson equation whose small ε means heat spreads only a short distance laterally before it is removed vertically. This is why a few sensors cannot see a hot block far from them, and why the problem is hard at 5 sensors.

Metrics. All metrics are computed on non-sensor grid nodes only, in °C: relative L_2 error $\|\hat{T} - T\|_2 / \|T\|_2$, mean absolute error (MAE), root-mean-square error (RMSE), peak-temperature error $\max \hat{T} - \max T$, and hotspot localisation error, the Euclidean distance in grid cells between the predicted and true maxima. For the real maps, whose temperatures sit far above zero, we also report the error relative to the temperature *rise*, $\|\hat{T} - T\|_2 / \|T - \min T\|_2$, which is not inflated by the large mean.

IV. Data

A. Simulated dies

We generated 700 dies of 12×12 mm with a HotSpot-style solver [24]. Each die holds 2 to 8 non-overlapping rectangular blocks of four types (CPU, GPU, cache, memory controller) with dynamic power densities drawn from type-specific ranges between 5 and 150 W/cm^2 and leakage fractions of 3–5%, with leakage coefficient $\beta = 0.012\,\mathrm{K}^{-1}$. Ambient temperature is uniform in $[25, 45]$ °C, $t_{\mathrm{die}} = 500\,\mu$m and $h_{\mathrm{eff}} = 6256$ W/(m^{2}K). Equation (1) is solved by Picard iteration (inner loop for $k(T)$, outer loop for leakage) on a 512×512 mesh and downsampled to 128×128, so the reference maps are not produced by the discretisation any model uses, which avoids an "inverse crime" [26]. The dies are split 500/100/100 into training, validation and test sets; floorplans are never shared between splits. [TODO: Confirm t_{die}, h_{eff} and the power ranges; PROTOCOL.md marks them as assumptions not yet traced to a source.]

B. Real IR maps

We use the public dataset of Lu and Tan [13], which records IR maps of commercial processors while they run benchmark workloads. We keep the two Intel CPUs, i5-3337U (177×166 pixels, 5 workloads) and i7-8650U (185×154 pixels, 19 workloads), each with 100 consecutive frames per workload. The Ryzen 7 4800U and RTX 4060 recordings were excluded because their maps contain a fixed pattern (printed text and a

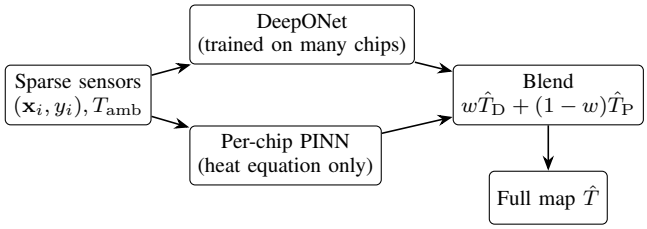


Fig. 1. Hybrid reconstruction. A DeepONet learned from many chips and a PINN fitted to one chip's sensors and the heat equation each predict the full map; their predictions are blended with a weight w chosen on validation chips. The physics-informed DeepONet instead adds the heat-equation residual to the DeepONet's training loss.

grid structure) that does not change with the thermal state, an emissivity artefact rather than temperature. Consecutive frames are nearly identical, so splits are made by whole workload: i5 uses 3/1/1 workloads (300/100/100 frames) for training, validation and test; i7 uses 13/3/3 workloads (1300/300/300 frames). The performance counters in the dataset are not used.

C. Sensors and noise

Sensors are placed by POD-QR [15]: the training maps (normalised per sample for the simulated set) are centred, the leading POD modes holding 95% of the energy are kept (11 modes for the simulated set), and QR factorisation with column pivoting on the transposed mode matrix ranks the grid points. The first 5, 8 and 10 pivots, excluding a 3-cell border, give nested layouts ($5 \subset 8 \subset 10$). Layouts are fitted on training data only and fixed for all chips. Each reading is the true temperature plus a per-sensor calibration offset $\sim \mathcal{N}(0, 0.5^2)\,^\circ$C and Gaussian noise $\sigma = 1\,^\circ$C, then quantised to $1\,^\circ$C, a model of a typical DTS. Models receive only these noisy readings; the true values are used only for scoring. The same sensor model is applied to the real IR maps, where the "sensors" are pixels of the IR image.

V. Methods

Fig. 1 summarises the models. All networks use PyTorch and Adam [27] with cosine learning-rate decay, keep the checkpoint with the lowest validation error, and are scored once on the test set.

A. Data-driven DeepONet

The DeepONet [4] predicts

$$\hat{\theta}(\mathbf{x}) = \mathbf{b}(\mathbf{s})^\top \mathbf{t}(\mathbf{x}) + b_0, \tag{3}$$

where the branch network $\mathbf{b}$ receives the sensor vector $\mathbf{s} = [x_1, y_1, \theta_1, \ldots, x_N, y_N, \theta_N]$ (positions and readings) and the trunk network $\mathbf{t}$ receives a query coordinate in $[0,1]^2$. Both are tanh multilayer perceptrons of width 128 (3 branch and 4 trunk hidden layers) with a $p = 128$-dimensional output, about 1.2×10^5 parameters in all. Temperatures are scaled as $\theta = (T - T_{\mathrm{amb}})/S$ with one global S, the largest rise in the training set. Training minimises the mean squared error on 1024 random grid points per chip, with batches of 32 chips, learning rate 10^{-3} decaying to 10^{-5}, up to 400 epochs and early stopping on validation error. One network is trained per sensor count, for 5 seeds. On the real maps the same network is trained for 6000 steps with the same hyperparameters.

B. Per-chip PINN

The PINN uses no training set. For each test chip a tanh network of 4 layers by 64 units represents $\theta(\mathbf{x})$; it receives $(\cos \pi x, \cos \pi y)$ instead of (x, y), which makes $\partial\theta/\partial n = 0$ on all four edges for any weights, so the adiabatic boundary condition holds exactly. Because the power map is unknown, the dynamic source is learned as a sum of 6 Gaussian blobs with trainable amplitudes, centres and widths, and leakage is applied on top as $1 + f \exp(\beta(T - 25))$ with the mean leakage fraction $f = 0.0425$. The loss is

$$\mathcal{L} = \frac{1}{M}\sum_{j=1}^{M} R(\mathbf{x}_j)^2 + \lambda_{\mathrm{d}} \frac{1}{N}\sum_{i=1}^{N} \big(\hat{\theta}(\mathbf{x}_i) - \theta_i\big)^2, \tag{4}$$

with R the residual of (2) computed by automatic differentiation at $M = 2048$ random collocation points and $\lambda_{\mathrm{d}} = 100$. Optimisation runs 5000 Adam steps followed by 2000 L-BFGS steps; one solve takes about 3 minutes on a 4-core CPU. We report one seed per chip.

C. Hybrid 1: physics-informed DeepONet

This model adds physics as a training loss [11]. The DeepONet gains a second output, a non-negative dynamic power density $\hat{q}$, computed from the same branch embedding; the temperature trunk receives $(\cos \pi x, \cos \pi y)$ for exact adiabatic edges. The heat-equation residual linking $\hat{\theta}$ and $\hat{q}$ is evaluated at 64 collocation points per chip and added to the data loss with weight 0.1, ramped up from zero over the first 40 epochs. The power density is never supervised, so the physics term needs no extra data. An optional second stage refines each test chip with the physics loss; we report it as an ablation.

D. Hybrid 2: validation-weighted blend

The blend combines the two independent estimators in temperature space,

$$\hat{T}_{\mathrm{blend}} = w\,\hat{T}_{\mathrm{DeepONet}} + (1 - w)\,\hat{T}_{\mathrm{PINN}}, \tag{5}$$

with $w \in \{0, 0.1, \ldots, 1\}$ chosen per sensor count by the mean relative L_2 error on the 100 *validation* chips, then fixed and applied once to the test chips. Because $w = 0$ and $w = 1$ are allowed, the blend can always fall back on the better component, and w itself measures how much the learned operator is trusted. The blend uses the seed-0 DeepONet.

E. Real IR maps: physics reconstruction and correction hybrid

For the real maps the power map, die stack and material constants are unknown, so we use a physics-based reconstruction with the structure of (1) but linearised: $(1 - \ell^2\nabla^2)(T - T_0) = q/h$ with adiabatic edges and a decay length ℓ in pixels. The unknown source is expanded in Gaussian bumps on a coarse grid; the response of each bump is computed with a discrete cosine transform (which enforces the adiabatic edges), and

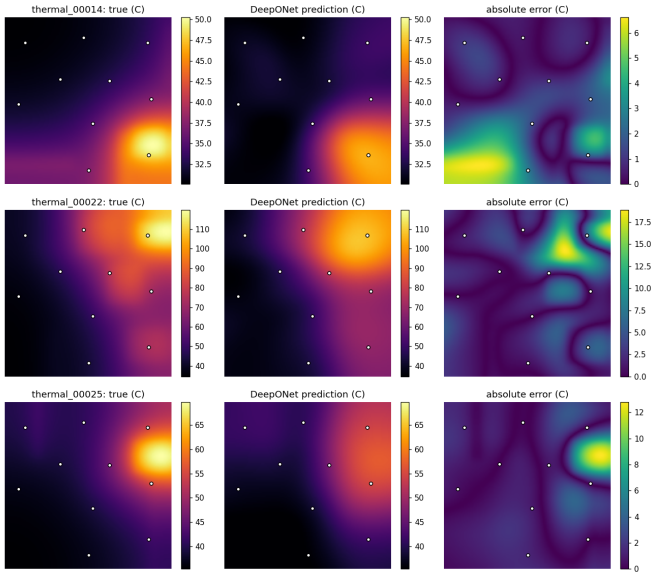


Fig. 2. DeepONet reconstructions of three simulated test chips with 10 sensors (white dots), seed 0. Left: reference map; centre: prediction; right: absolute error (°C). Errors concentrate on blocks that lie between sensors.

the bump amplitudes and the constant T_0 are fitted to the sensor readings by ridge regression. Like a PINN, it uses no training maps; its three settings (ℓ, bump spacing, ridge weight) are chosen on validation frames. We test three hybrids, with every setting chosen on validation frames: (i) the blend above; (ii) a *correction*, which fits the same physics response to the DeepONet's residual at the sensors, $y_i - \hat{T}_{\mathrm{D}}(\mathbf{x}_i)$, and adds it to the DeepONet map, spreading the sensor mismatch in a way the heat equation allows; and (iii) blend followed by correction. Variant (ii) had the lowest mean validation error over all settings and is reported as "the hybrid".

F. Classical baselines

Gappy POD [14] fits the readings with the leading POD modes of the training maps (number of modes chosen on validation data). The *training-mean map* ignores the sensors and is the error floor of a model that learns nothing.

VI. Results on Simulated Data

A. Accuracy with 500 training chips (RQ1)

Table I lists test errors on the 100 test chips. The number of training seeds is given for every row; where several seeds exist the table shows mean ± standard deviation over seeds. The DeepONet reaches 11.5%, 8.6% and 6.1% relative L_2 error at 5, 8 and 10 sensors (MAE 4.7, 3.4 and 2.4 °C), and reconstructs one 128×128 map in about 40–50 ms on a CPU, roughly $4000\times$ faster than a per-chip PINN solve. Error falls steeply with sensor count, as expected from the short lateral heat-spreading length in (2). Fig. 2 shows typical reconstructions: the DeepONet finds the main hot block, but blocks between sensors are smoothed or missed, which produces the largest errors.

B. Physics as a loss term (RQ2)

Adding the heat-equation residual to the DeepONet's loss leaves the error unchanged to within the seed spread at every sensor count (Table I, 11.47 vs 11.46%, 8.61 vs 8.62%, 6.16 vs 6.13%), and refining each test chip with the physics loss makes it slightly worse. With 500 densely supervised training maps the data term already pins down the field, and the residual term has little left to constrain.

C. Physics as a separate estimator (RQ2)

The per-chip PINN, which sees nothing but one chip's sensors and the heat equation, is worse than the DeepONet at 5 sensors (13.3 vs 11.5%) and close to it at 8 and 10 sensors. It is, however, much better at the quantity that matters most for thermal protection, the peak temperature (Table II): the DeepONet under-predicts the peak by 7.4 to 13.8 °C, the PINN by only 2.3 to 8.0 °C. A data-driven model regresses towards the average chip and flattens rare, intense hot spots; the PINN's source model and the physics of (1) let it extrapolate the peak from nearby readings. The PINN is worse at locating the hotspot, because its smooth Gaussian sources cannot reproduce sharp block edges.

The two estimators make different errors, so combining them helps. The blend has the lowest error at 8 and 10 sensors and the lowest MAE and RMSE at 8 and 10 sensors (Table I). Table III gives paired differences per chip with 95% bootstrap confidence intervals. The blend is significantly better than the PINN at every sensor count, and better than the DeepONet at 8 sensors (+0.38 points, CI $[0.13, 0.62]$, better on 67% of chips); at 5 and 10 sensors the difference from the DeepONet is not significant. With 500 training chips the selected weights are $w = 0.8$, 0.5 and 0.5: the validation data themselves say that the PINN deserves a real share once 8 or more sensors are available.

D. Data scarcity (RQ2)

The central question for a physics prior is what happens when data are scarce. We retrained the DeepONet on random subsets of 25, 50 and 100 training chips (5 seeds each) and re-selected the blend weight on validation chips; the PINN needs no training chips, so it is unaffected. Fig. 3 and Table IV show the result. The DeepONet degrades quickly as data shrink (6.1% to 9.3% at 10 sensors, 8.6% to 12.7% at 8 sensors), while the blend stays close to or below the PINN at every size. With 25 training chips the blend lowers the error by 3.9 points at 8 sensors (12.7 to 8.8%) and by 2.9 points at 10 sensors (9.3 to 6.4%), relative reductions of 31% and 31%, all with confidence intervals well clear of zero. The selected weight moves accordingly: at 25 chips the DeepONet receives only $w = 0.1$ to 0.3, at 500 chips 0.5 to 0.8. The blend is never worse than its better component by more than 0.2 points in any of the 12 settings, and better than both in 11 of them (all except 5 sensors with 500 chips).

Two observations temper this result. First, at 25 and 50 chips most of the blend's gain comes from the PINN itself, which is nearly as good on its own (8.80 vs 8.78% at 8

TABLE I
SIMULATED TEST SET (100 CHIPS, 500 TRAINING CHIPS). RELATIVE L_2 ERROR (%) AND MAE (°C) ON NON-SENSOR NODES; LOWER IS BETTER.

		Rel. L_2 (%)			MAE (°C)		
Model	Seeds	N=5	N=8	N=10	5	8	10
DeepONet	5	**11.46**±.10	8.62±.06	6.13±.03	4.66	3.37	2.38
Phys.-inf. DeepONet	5	11.47±.10	8.61±.06	6.16±.06	4.67	3.36	2.39
+ per-chip refine	1	11.88	8.94	6.78	4.76	3.49	2.69
Per-chip PINN	1	13.33	8.80	6.51	5.19	3.30	2.39
Blend (DeepONet+PINN)	1	11.62	**8.21**	**6.01**	4.68	**3.16**	**2.27**

TABLE II
PEAK-TEMPERATURE ERROR (°C, NEGATIVE = UNDER-PREDICTION) AND HOTSPOT LOCALISATION ERROR (GRID CELLS) ON THE 100 TEST CHIPS. ONE SEED EACH (DEEPONET SEED 0).

	Peak error (°C)			Hotspot error (cells)		
Model	N=5	8	10	5	8	10
DeepONet	−13.8	−9.0	−7.4	35.3	**27.4**	**18.2**
Per-chip PINN	−8.0	−3.7	−2.3	46.5	32.7	27.3
Blend	−13.3	−7.2	−5.5	**34.9**	28.5	22.3

TABLE III
PAIRED COMPARISON OF THE BLEND ON THE 100 TEST CHIPS (500 TRAINING CHIPS): MEAN REDUCTION OF RELATIVE L_2 ERROR IN PERCENTAGE POINTS (POSITIVE = BLEND BETTER), 95% BOOTSTRAP CI, AND SHARE OF CHIPS WHERE THE BLEND IS BETTER.

N	vs	Gain (pts)	95% CI	Chips better
5	DeepONet	−0.03	$[-0.24, 0.16]$	63%
	PINN	+1.71	$[0.82, 2.65]$	59%
8	DeepONet	+0.38	$[0.13, 0.62]$	67%
	PINN	+0.59	$[0.22, 1.03]$	51%
10	DeepONet	+0.15	$[-0.09, 0.36]$	59%
	PINN	+0.50	$[0.24, 0.79]$	69%

sensors with 25 chips); the value of the blend is that the validation-chosen weight finds the right mix automatically in every regime, without knowing in advance which component is better. Second, at 5 sensors no method is good: all errors lie between 11.5 and 13.8%, which suggests that 5 sensors carry too little information about blocks far from them, whatever the model.

VII. RESULTS ON REAL IR MAPS

Table V reports test errors on the held-out workloads of both processors (RQ3). Both learned models recover the maps to 0.8–2.4 °C MAE, far better than the training-mean map (6.1 and 10.4 °C), and the error falls with sensor count. The physics reconstruction alone is clearly weaker than the DeepONet (rise error 0.16–0.50 against 0.10–0.25), because a linear model with a short decay length and a few bumps cannot represent a whole real map from 5 to 10 readings without training data. Used as a correction of the DeepONet, it helps: paired over test frames, the hybrid is better than the DeepONet at 10 sensors on the i5 and at 8 and 10 sensors on the i7 (rise-error reductions of 3 to 11%, CIs clear of zero), ties at i5/8 and i7/5, and is worse only at i5/5. It is better than the physics reconstruction in all six settings.

TABLE IV
DATA-SCARCITY STUDY: TEST RELATIVE L_2 ERROR (%) ON 100 TEST CHIPS. DEEPONET: MEAN ± S.D. OVER 5 SEEDS. PINN: 1 SEED, NO TRAINING CHIPS. BLEND: DEEPONET SEED 0 WITH THE PINN; w = DEEPONET WEIGHT CHOSEN ON VALIDATION; LAST COLUMN: PAIRED GAIN OF THE BLEND OVER DEEPONET SEED 0 (POINTS, 95% CI).

N	Chips	DeepONet	PINN	Blend	w	Gain [95% CI]
5	25	13.83±.10	13.33	**12.76**	0.3	+1.08 [0.51, 1.65]
	50	13.34±.12	13.33	**12.51**	0.4	+0.95 [0.33, 1.56]
	100	12.44±.15	13.33	**12.21**	0.6	+0.39 [−0.03, 0.80]
	500	**11.46**±.10	13.33	11.62	0.8	−0.03 [−0.24, 0.17]
8	25	12.74±.08	8.80	**8.78**	0.1	+3.87 [3.15, 4.61]
	50	11.34±.32	8.80	**8.72**	0.2	+2.36 [1.75, 2.99]
	100	9.52±.09	8.80	**8.53**	0.4	+0.87 [0.54, 1.18]
	500	8.62±.06	8.80	**8.21**	0.5	+0.38 [0.14, 0.62]
10	25	9.33±.12	6.51	**6.44**	0.1	+2.86 [2.26, 3.50]
	50	7.26±.08	6.51	**6.23**	0.3	+0.97 [0.64, 1.29]
	100	6.42±.05	6.51	**6.11**	0.4	+0.31 [0.04, 0.57]
	500	6.13±.03	6.51	**6.01**	0.5	+0.15 [−0.08, 0.37]

The classical gappy POD is a strong baseline on these data. On the i5 it is the best method at every sensor count, and on the i7 it is the best at 10 sensors, although the DeepONet and the hybrid are far better at 5 and 8 sensors (rise error 0.10 vs 0.21 at 8 sensors). The real maps explain this: each is a single smooth, camera-blurred hot region, so a few POD modes describe it well, and with enough sensors a linear fit pins it down. The i5 also has only three training workloads, too few for a network to learn much beyond its POD modes. Fig. 4 shows one typical test frame per processor: on the i7 the DeepONet places the hot region correctly while gappy POD misplaces it; on the i5 both are close.

VIII. OTHER PHYSICS-INFORMED ARCHITECTURES

To check that the conclusions are not specific to the DeepONet, we trained further architectures on the simulated set under the same protocol (Table VI). All of them learn across chips and predict both temperature and source from an encoding of the sensor readings, with a physics residual weighted 0.01 unless stated: an FNO [21] with a physics loss (PINO [12]; 4 Fourier layers, 8 modes, 5.3×10^5 parameters), an FBPINN [22] with 9 overlapping, smoothly blended windows, and an XPINN [23] with 3×3 tiles coupled by interface losses (both about 2.4×10^4 parameters), and a hybrid that predicts the source and obtains temperature from a differentiable grid solver of (1), compared with an otherwise identical data-only twin. These runs had smaller budgets and mostly one seed, so the table is a survey rather than a ranking.

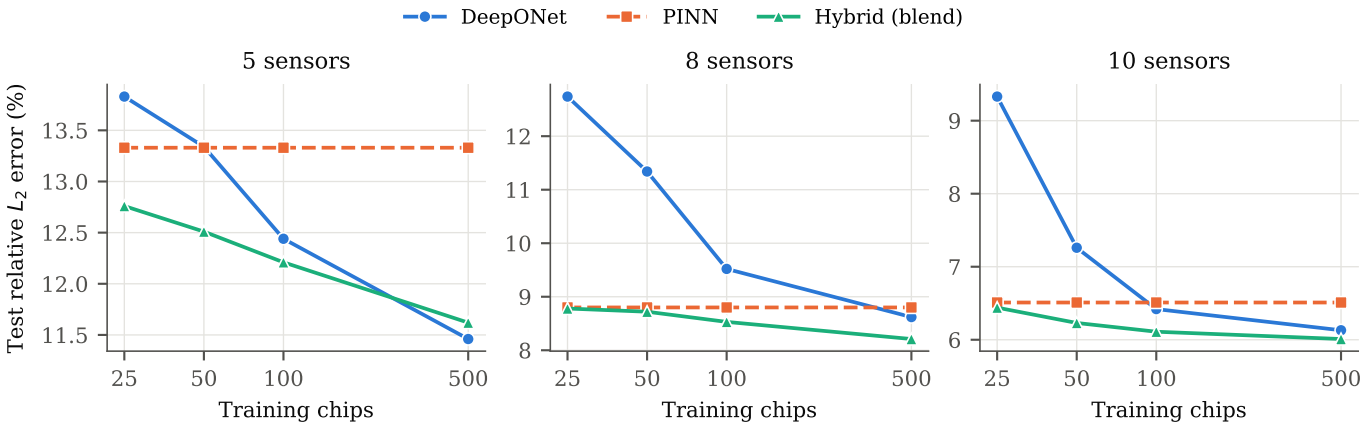


Fig. 3. Test relative L_2 error against the number of training chips. The DeepONet (5-seed mean) degrades as data shrink; the per-chip PINN needs no training chips; the validation-weighted blend follows the better of the two and improves on both in most settings.

TABLE V
REAL IR MAPS, HELD-OUT TEST WORKLOADS. ERROR RELATIVE TO THE TEMPERATURE RISE (LOWER IS BETTER) AND MAE (°C). DEEPONET AND HYBRID: MEAN ± S.D. OVER 5 SEEDS. PHYSICS RECONSTRUCTION AND GAPPY POD ARE DETERMINISTIC.

		Rise error			MAE (°C)		
Model	Seeds	N=5	8	10	5	8	10
Intel i5-3337U (test: 1 workload, 100 frames)							
DeepONet	5	.227±.022	.152±.013	.149±.017	1.22	0.80	0.79
Physics recon.	–	.419	.248	.158	2.21	1.33	0.83
Hybrid (correction)	5	.256±.037	.149±.018	.141±.018	1.35	0.79	0.76
Gappy POD	–	**.169**	**.113**	**.129**	**0.91**	**0.62**	**0.69**
Training-mean map	–	.967	.967	.967	6.07	6.07	6.07
Intel i7-8650U (test: 3 workloads, 300 frames)							
DeepONet	5	**.246**±.005	.104±.009	.111±.045	2.42	1.02	1.09
Physics recon.	–	.501	.332	.284	5.16	3.50	2.94
Hybrid (correction)	5	**.246**±.007	**.101**±.007	.099±.027	**2.41**	**1.00**	0.98
Gappy POD	–	.367	.214	**.086**	3.86	2.26	**0.85**
Training-mean map	–	.892	.892	.892	10.43	10.43	10.43

TABLE VI
FURTHER ARCHITECTURES ON THE SIMULATED TEST SET (500 TRAINING CHIPS UNLESS STATED). RELATIVE L_2 ERROR (%). SEEDS LISTED PER SENSOR COUNT; MULTI-SEED ENTRIES ARE MEAN ± S.D.

Model	Seeds	N=5	N=8	N=10
DeepONet (reference)	5/5/5	11.46	8.62	6.13
FNO/PINO	1/1/3	13.20	10.79	9.78±.20
FBPINN, physics on	1/3/3	11.35	8.95±.05	6.76±.03
FBPINN, physics off	–/–/1	–	–	6.83
XPINN, physics on*	1/1/3	11.43	8.72	6.45±.03
XPINN, physics off	1/1/1	11.38	8.63	6.40
Solver hybrid / data-only twin	–/–/1	–	–	6.58 / 6.53
50 training chips	–/–/1	–	–	10.81 / 9.79
25 training chips	–/–/1	–	–	12.58 / 11.48

*Physics residual masked next to tile edges; without the mask: 16.2% at N=10.

Three findings stand out. First, no architecture beats the DeepONet; the domain-decomposition models come within 0.3–0.6 points at 10 sensors with a fifth of its parameters. Second, switching the physics term on or off changes the error by at most 0.07 points (FBPINN 6.76 vs 6.83%, XPINN 6.45 vs 6.40%), and in a data-scarcity test with the XPINN (25 and 50 chips) by at most 0.15 points. Third, physics in the loss can hurt: the XPINN stalled at 16% error until the residual was masked next to tile edges, where the solution jumps; the FNO became worse with a larger physics weight (13.6 vs 10.4% at 3000 steps with weight 0.1); and the solver hybrid, which forces every prediction through the physics, was 1 point *worse* than its data-only twin with 25 or 50 training chips. A data-driven variant of the PINN trained across all 500 chips showed the same small effect in a short preliminary run (6.23 vs 6.45% with and without physics, one seed).

Why physics in the loss adds little. Passing the reference maps themselves through a 128×128 finite-volume discretisation of (1) leaves a pointwise residual of 32.9% of the signal (40 validation chips); 99% of its squared magnitude sits in the worst 5% of pixels, at hot spots and sharp block edges, and it falls to 7.0% away from block edges and to 7.9% when averaged over 16×16 patches. The references come from a finer mesh and sharp, block-wise sources that a coarse grid operator cannot reproduce exactly. A PDE loss on the coarse grid therefore asks the network to move away from the very maps it is fitted to, exactly where the error is largest. The per-chip PINN and the blend avoid this conflict: the physics model makes its own prediction, and the validation data decide how much of it to keep.

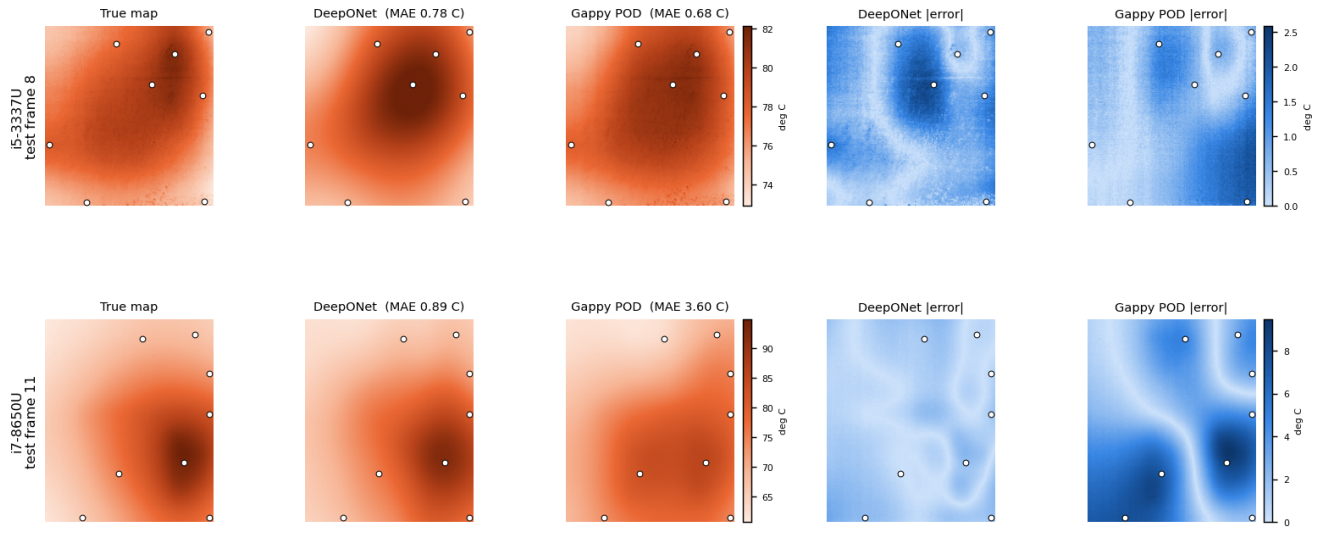


Fig. 4. Real IR maps with 8 sensors (white dots), one typical test frame per processor (seed 0). From left: measured map, DeepONet, gappy POD (the three share one temperature scale), and the absolute errors of DeepONet and gappy POD.

The architectures in Table VI were also run on the i7 IR maps, but with a different split (by mean temperature, test workloads fio, encode-flac and t-test1) and one seed, so the numbers are not comparable with Table V. On that split a data-only decoder, FBPINN, XPINN and FNO reached 2.5–4.2% relative L_2 error (absolute temperature) at 10 sensors, against 2.4% for thin-plate-spline interpolation and 2.0% for a POD fit, repeating the observation that classical methods are competitive on smooth real maps when 10 sensors are available. The per-chip PINN was likewise tested on 27 IR frames from 9 devices as a cross-domain check (4.5%, 2.3% and 1.8% relative L_2 at 5, 8 and 10 sensors, 3 seeds), again with a separate protocol.

IX. Discussion

A. When does physics help?

Our results give a consistent answer across models and datasets. Physics added as a *loss term* to a model trained on hundreds of supervised maps is neutral at best and harmful when the PDE and the reference data disagree at grid scale. Physics used as a *separate, data-free estimator* helps in two ways: it carries information that the training set lacks when training data are scarce, and it is biased differently from a learned operator, notably recovering the peak temperature that data-driven models flatten. A simple validation-weighted combination exploits both and degrades gracefully as data shrink, which is the property a deployed virtual sensor needs. For the real maps, where the physics is known only in form, using it to correct the learned prediction at the sensors works better than blending in a weak physics estimate.

B. Implications for thermal measurement

For a design-time or run-time thermal monitor, three practical points follow. (i) A trained DeepONet produces a full map in tens of milliseconds, fast enough for run-time use, whereas a per-chip PINN takes minutes and suits offline analysis. (ii) The peak temperature is systematically under-estimated by learned models; a physics-based estimate or correction should be part of any throttling decision. (iii) A strong classical baseline (gappy POD, spline interpolation) must be reported: on smooth real maps with 10 sensors it can match a network.

C. Limitations

The simulated dies use a two-dimensional compact model with uniform power per block and several constants (t_{die}, h_{eff}, power ranges) that are plausible but not traced to a specific product. Sensor positions are fixed per study by POD-QR; robustness to other placements was not tested. Several entries rest on one seed (the PINN, the blend, most architectures in Table VI); we report the seed count for every entry and give paired confidence intervals for the main claims. The real-data study covers two processors; the i5 has a single validation and a single test workload, the IR maps are smooth and camera-blurred, and the variant of the real-data hybrid was fixed after some per-seed test numbers had been seen. The models of Section VIII were run on the real maps with a different split from Table V. Finally, only steady state is modelled.

X. Conclusion and Future Work

We studied full-chip thermal map reconstruction from 5 to 10 noisy sensors as a neural-network learning problem and asked when the heat equation helps. A data-driven DeepONet reconstructs simulated maps to 6–11% relative error in milliseconds, and adding the PDE residual to its loss, or to FNO, FBPINN and XPINN models, gives no measurable gain. Combining the DeepONet with an independent per-chip PINN through a validation-chosen weight is never worse than its better component, improves accuracy and peak-temperature estimates, and helps most when training data are scarce, cutting the error by about a third with 25 training chips. On real

IR maps of two Intel CPUs, a physics-based correction of the DeepONet improves on it in half of the settings, while gappy POD remains a strong baseline. Future work includes transient reconstruction, learning the sensor placement jointly with the reconstruction, uncertainty estimates for the blended map, three-dimensional die-and-package models, and validation on more processors with measured power.

Data and Code Availability

[TODO: Repository URL and licence. The real IR maps are not redistributed; they are available from the original authors [13].]

Acknowledgment

[TODO: Funding and acknowledgements.]

References

[1] A. N. Nowroz, R. Cochran, and S. Reda, "Thermal monitoring of real processors: Techniques for sensor allocation and full characterization," in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2010, pp. 56–61.

[2] J. Ranieri, A. Vincenzi, A. Chebira, D. Atienza, and M. Vetterli, "EigenMaps: Algorithms for optimal thermal maps extraction and sensor placement on multicore processors," in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2012, pp. 636–641.

[3] S. Sadiqbatcha, J. Zhang, H. Amrouch, and S. X.-D. Tan, "Real-time full-chip thermal tracking: A post-silicon, machine learning perspective," *IEEE Transactions on Computers*, vol. 71, no. 6, pp. 1411–1424, 2022.

[4] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.

[5] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[6] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[7] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, "Scientific machine learning through physics-informed neural networks: Where we are and what's next," *Journal of Scientific Computing*, vol. 92, no. 3, p. 88, 2022.

[8] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, p. 66, 2022.

[9] S. Cai, Z. Wang, S. Wang, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks for heat transfer problems," *ASME Journal of Heat Transfer*, vol. 143, no. 6, p. 060801, 2021.

[10] Z. Du and R. Lu, "Physics-informed neural networks for advanced thermal management in electronics and battery systems: A review of recent developments and future prospects," *Batteries*, vol. 11, no. 6, p. 204, 2025.

[11] S. Wang, H. Wang, and P. Perdikaris, "Learning the solution operator of parametric partial differential equations with physics-informed DeepONets," *Science Advances*, vol. 7, no. 40, p. eabi8605, 2021.

[12] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar, "Physics-informed neural operator for learning partial differential equations," *ACM/IMS Journal of Data Science*, vol. 1, no. 3, 2024.

[13] J. Lu and S. X.-D. Tan, "Thermal map dataset for commercial multi/many core CPU/GPU/TPU," in *Proc. ACM/IEEE Int. Symp. Machine Learning for CAD (MLCAD)*, 2024, pp. 1–7.

[14] R. Everson and L. Sirovich, "Karhunen–Loève procedure for gappy data," *Journal of the Optical Society of America A*, vol. 12, no. 8, pp. 1657–1664, 1995.

[15] K. Manohar, B. W. Brunton, J. N. Kutz, and S. L. Brunton, "Data-driven sparse sensor placement for reconstruction: Demonstrating the benefits of exploiting known patterns," *IEEE Control Systems Magazine*, vol. 38, no. 3, pp. 63–86, 2018.

[16] W. Jin, S. Sadiqbatcha, J. Zhang, and S. X.-D. Tan, "Full-chip thermal map estimation for commercial multi-core CPUs with generative adversarial learning," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2020, pp. 1–9.

[17] L. Chen, J. Lu, W. Jin, and S. X.-D. Tan, "Fast full-chip parametric thermal analysis based on enhanced physics enforced neural networks," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023, pp. 1–8.

[18] H. Son, H. Cho, and H. J. Hwang, "Physics-informed neural networks for microprocessor thermal management model," *IEEE Access*, vol. 11, pp. 122 974–122 979, 2023.

[19] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. A3055–A3081, 2021.

[20] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 26 548–26 560, 2021.

[21] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[22] B. Moseley, A. Markham, and T. Nissen-Meyer, "Finite basis physics-informed neural networks (FBPINNs): a scalable domain decomposition approach for solving differential equations," *Advances in Computational Mathematics*, vol. 49, no. 4, p. 62, 2023.

[23] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," *Communications in Computational Physics*, vol. 28, no. 5, pp. 2002–2041, 2020.

[24] W. Huang, S. Ghosh, S. Velusamy, K. Sankaranarayanan, K. Skadron, and M. R. Stan, "HotSpot: A compact thermal modeling methodology for early-stage VLSI design," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 14, no. 5, pp. 501–513, 2006.

[25] C. J. Glassbrenner and G. A. Slack, "Thermal conductivity of silicon and germanium from 3 K to the melting point," *Physical Review*, vol. 134, no. 4A, pp. A1058–A1069, 1964.

[26] A. Wirgin, "The inverse crime," arXiv:math-ph/0401050, 2004.

[27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.